

Algorytmy klasyfikacji nienadzorowanej, laboratorium #3

Igor Józefowicz, 193257

Notatnik P1 (3 pkt) - klasyfikacja nadzorowana i nienadzorowana

Zadanie 1 - testowanie algorytmu kNN dla różnych wartości k

Uruchom algorytm k najbliższych sąsiadów dla kilku (około 3-4) wybranych wartości parametru k (rozpoczynając od 1).

1. Jaki charakter mają zmiany krzywej decyzyjnej wraz ze wzrostem tego parametru?
2. Czy dobór wartości tego parametru może potencjalnie wpływać na generalizację klasyfikatora kNN?

Kod

```
k_values = [1, 3, 5, 10, 15]

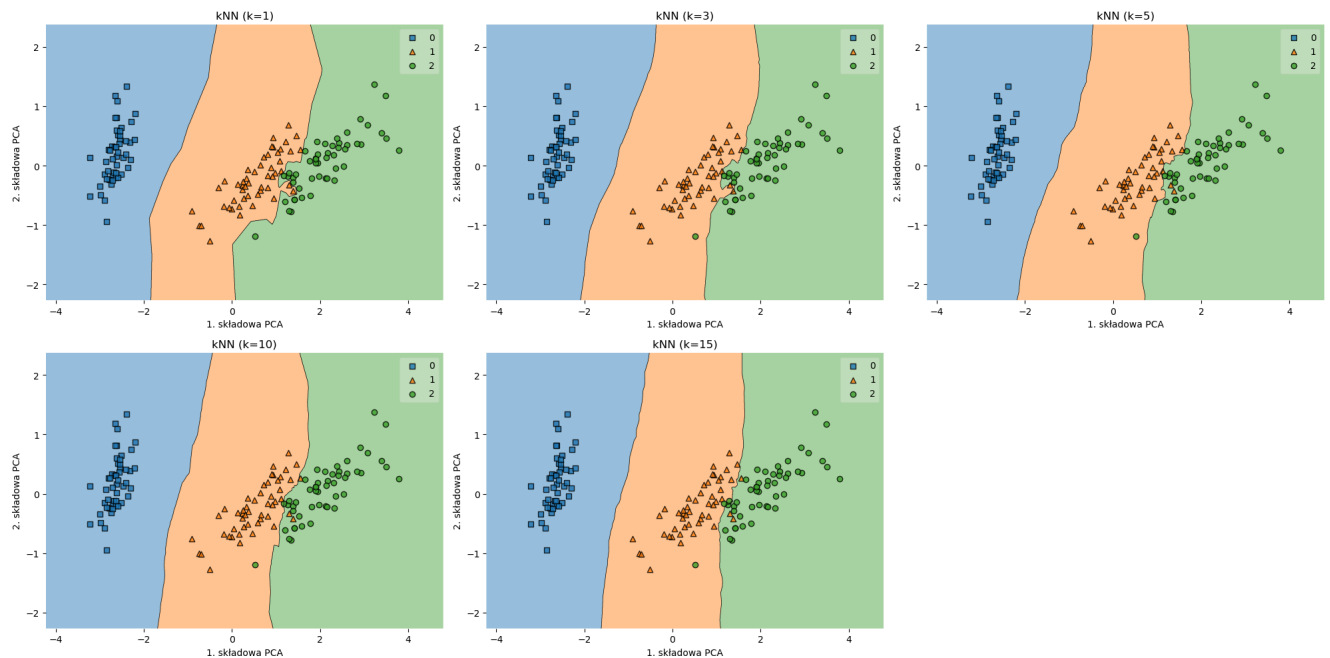
plt.figure(figsize=(20, 10))

for i, k in enumerate(k_values, start=1):
    plt.subplot(2, 3, i)
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(reduced_data, target)

    plot_decision_regions(reduced_data, target, clf=knn)
    plt.title(f'kNN (k={k})')
    plt.xlabel('1. składowa PCA')
    plt.ylabel('2. składowa PCA')

plt.tight_layout()
plt.show()
```

Wykresy



Wnioski do zadania 1:

1. Wraz ze wzrostem parametru k , granice decyzyjne stają się coraz bardziej wygładzone i uproszczone. Dla małych wartości (np. $k=1$) granica jest bardzo poszarpana. Ściśle dopasowuje się do poszczególnych punktów uczących. Z kolei dla większych k (np. 10 lub 15), skrajne pojedyncze obserwacje są ignorowane, traktowane jak szum. Wtedy granice mają bardziej regularny kształt.

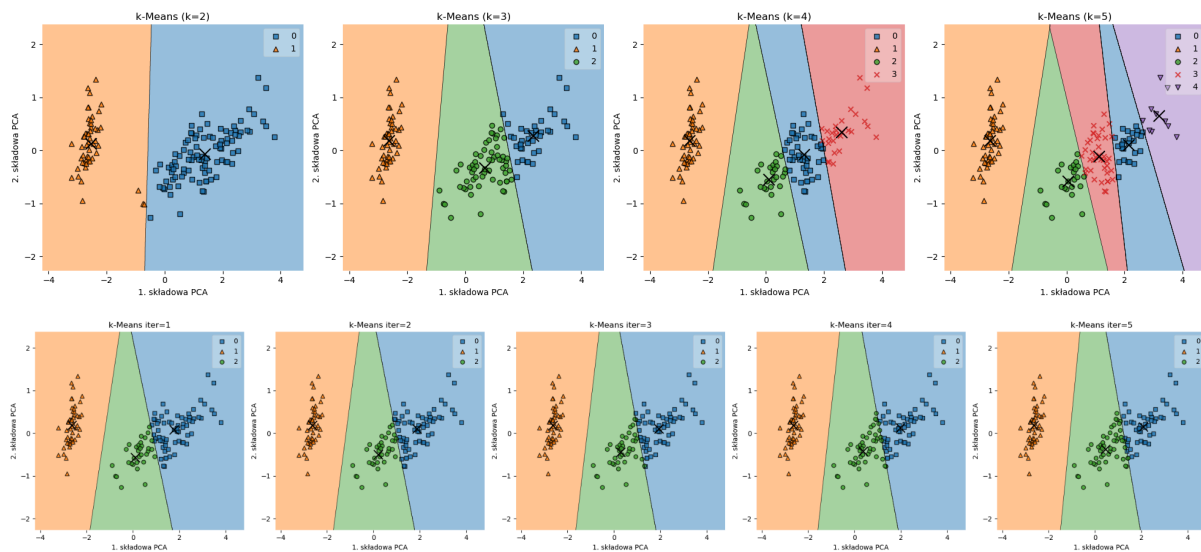
2. Tak, dobór parametru k zdecydowanie może wpływać na zdolność generalizacji modelu. Jeśli jest zbyt małe (np. $k=1$), model ma skłonność do przeuczenia się - dopasowuje się do szumu. Kiedy system operuje na większych k , model zyskuje lepszą zdolność do generalizacji → underfitting zaczyna się, gdy k jest zbyt duże i granice stają się za proste z pominięciem naturalnych wzorców.

Zadanie 2

Uruchom algorytm k średnich i zbadaj, jak wyglądają linie podziału biegnące pomiędzy klastrami dla różnych wartości parametru k . Zapisz też na dysku (za pomocą zakomentowanej linijki, dla $k=3$) wizualizacje efektów działania algorytmu dla kilku pierwszych iteracji algorytmu k średnich (zmieniając wartość zmiennej `num_iterations` np. od 1 do 5).

1. Opisz, jaki charakter mają zmiany następujące w położeniu klastrów (z iteracji na iterację)? W sprawozdaniu możesz, ale nie musisz zawierać ilustracji graficznej tych zmian (np. w postaci zrzutów ekranu).
2. Czy algorytm dobrze poradził sobie z "odgadnięciem" podziału na grupy jeżeli przyjmemy, że $k=3$? Odpowiedź uzasadnij.

Wykresy



Wnioski

1. Analizując kolejne iteracje algorytmu dla $k=3$, widać jak centroidy z początkowych, często nieoptymalnych pozycji przesuwają się w stronę poprawnego punktu środka dla każdej z grup punktów. Wraz z przemieszczaniem się centroidów, linie podziału (granice decyzyjne) również systematycznie się dostosowują, dążąc do stabilizacji.

2. Algorytm dobrze radzi sobie z bardzo dobrym wyodrębnieniem klasy Setosa, ponieważ odstaje ona znacznie od reszty zbioru → na wykresie po lewej stronie, widać dobrą separację.

Natomiast dla klas Versicolor i Virginica, które w zredukowanej przestrzeni silnie się przenikają i nie ma widocznej separacji, algorytm k-Means sztucznie kroi chmurę punktów w połowie, opierając się jedynie na odległości punktów. Powoduje to, że punkty leżące na pograniczu tych klas często lądują w niewłaściwym, sąsiednim klastrze. Podział nie jest więc w tym miejscu idealnie wierny rzeczywistej klasie.

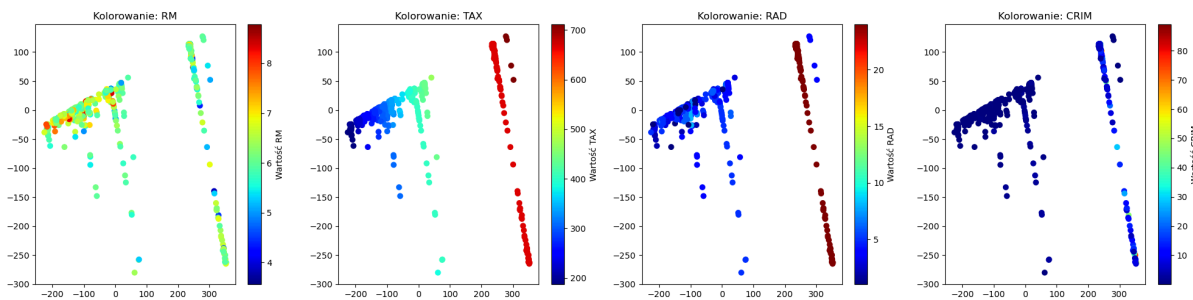
Notatnik P2 (2 pkt) - dobór liczby klas w algorytmie k średnich i klasteryzacji hierarchicznej

Zadanie 1

Zapoznaj się ze strukturą danych w zbiorze Boston Housing za pomocą wizualizacji na wykresie PCA (pierwsza wizualizacja od góry).

1. Zwróć uwagę, czy jesteś w stanie wizualnie wyróżnić odseparowane od siebie grupy punktów.
2. Zmieniając wartość zmiennej `coloring_variable_name` znajdź przykład 2 zmiennych (z wyjątkiem zmiennej `target`), które mogły mieć wpływ na to, że dane grupy punktów są od siebie odseparowane na wizualizacji. Podaj ich nazwy i wizualizacje w sprawozdaniu.

Wykresy



Wnioski

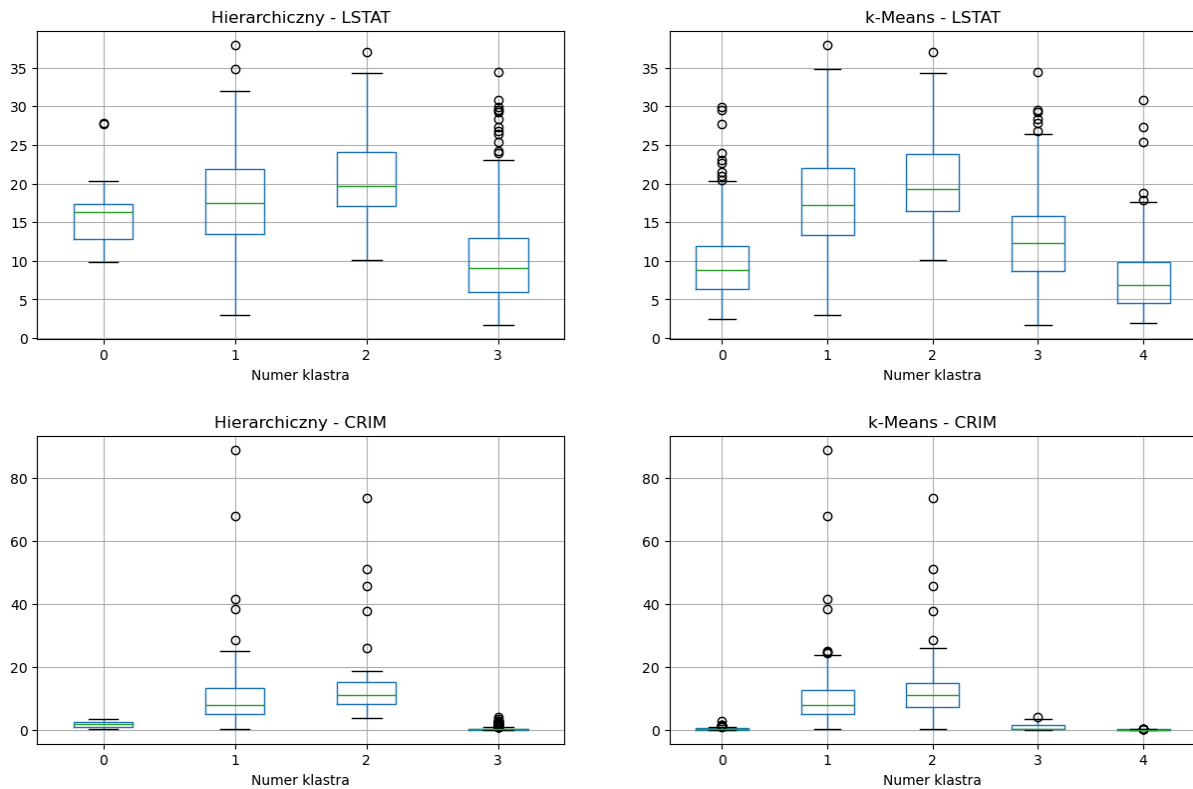
1. Zdecydowanie widać odseparowane chmury punktów na wykresie PCA. Główna część jest po prawej stronie, wyraźnie odcięta od tej chmury jest po lewej i na środku wykresu. Nienadzorowane metody szybko zauważają taki naturalny podział.

2. Zmienne, dzięki którym ten podział jest możliwy (między innymi), to **CRIM** (per capita crime rate by town) i **TAX** (full-value property-tax rate per \$10,000) → kolorowanie wyraźnie grupuje wartości duże w jednym rejonie, a małe w drugim. Podobnie działa cecha **RAD** → “index of accessibility to radial highways”. To między innymi te cechy decydują o wyglądzie wykresu PCA.

Zadanie 2

Przejdź do sekcji przykładu zawierającej wykresy pudełkowe. Pozostawiając domyślny zbiór danych (Boston Housing) sprawdź za pomocą wykresów pudełkowych, jak kształtują się wartości poszczególnych kolumn zbioru danych w różnych grupach wygenerowanych przez algorytmy klasyfikacji nienadzorowanej. Czy jesteś w stanie na tej podstawie wywnioskować, jakie czynniki miały wpływ na takie a nie inne ceny mieszkań w poszczególnych klasach? Podaj 2 przykłady zmiennych (z wyjątkiem zmiennej **target**), dla których można zaobserwować taką sytuację. Podaj ich nazwy i wykresy pudełkowe w sprawozdaniu. Wykonaj tę czynność zarówno dla algorytmu k średnich, jak i hierarchicznego.

Wykresy



Wnioski

Algorytmy klasteryzacji całkiem dobrze zgrupowały mieszkania o podobnej charakterystyce, co widać na przykład w zmiennych:

1. **CRIM** (per capita crime rate by town) → widać na wykresach pudełkowych, że jeden z klastrów niemal w całości zjada obszary o wysokim wskaźniku przestępczości, podczas gdy pozostałe utrzymują się blisko zera. Naturalnym wnioskiem jest to, że wysoka przestępczość obniża wartość nieruchomości.
2. **LSTAT** (% lower status of the population) → zauważalne są schodki między klastrami. Klasy mieszkań tańszych i średnich charakteryzują się wyraźnie wyższym poziomem LSTAT, co potwierdza, że status okolicy wpływa na podział i ostatecznie cenę. Występuje to w obu badanych algorytmach.

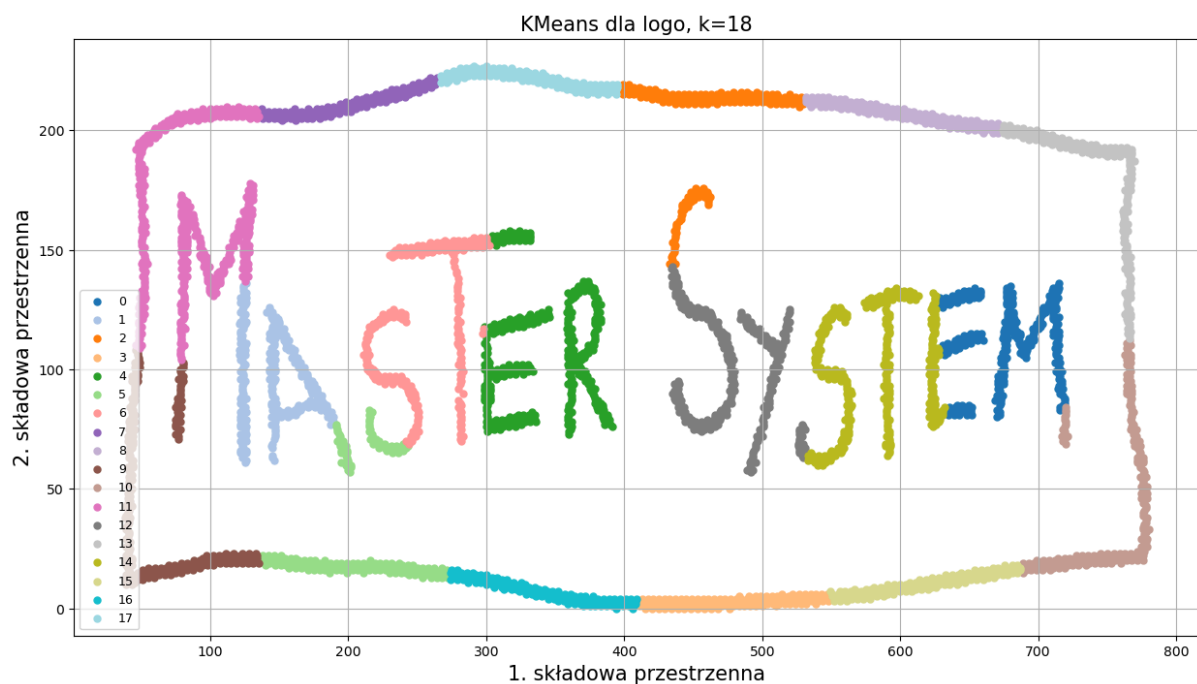
Notatnik P3 (3 pkt) - porównanie jakości segmentacji tekstu za pomocą algorytmu k średnich, hierarchicznego i bazującego na gęstości punktów (DBSCAN)

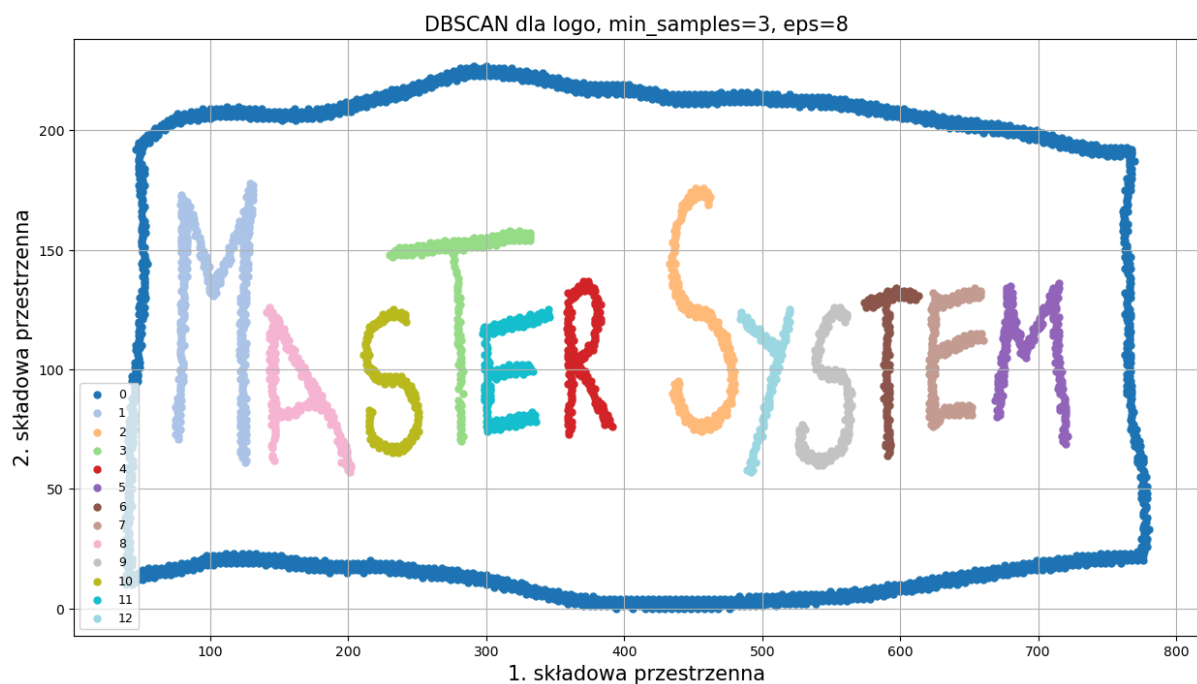
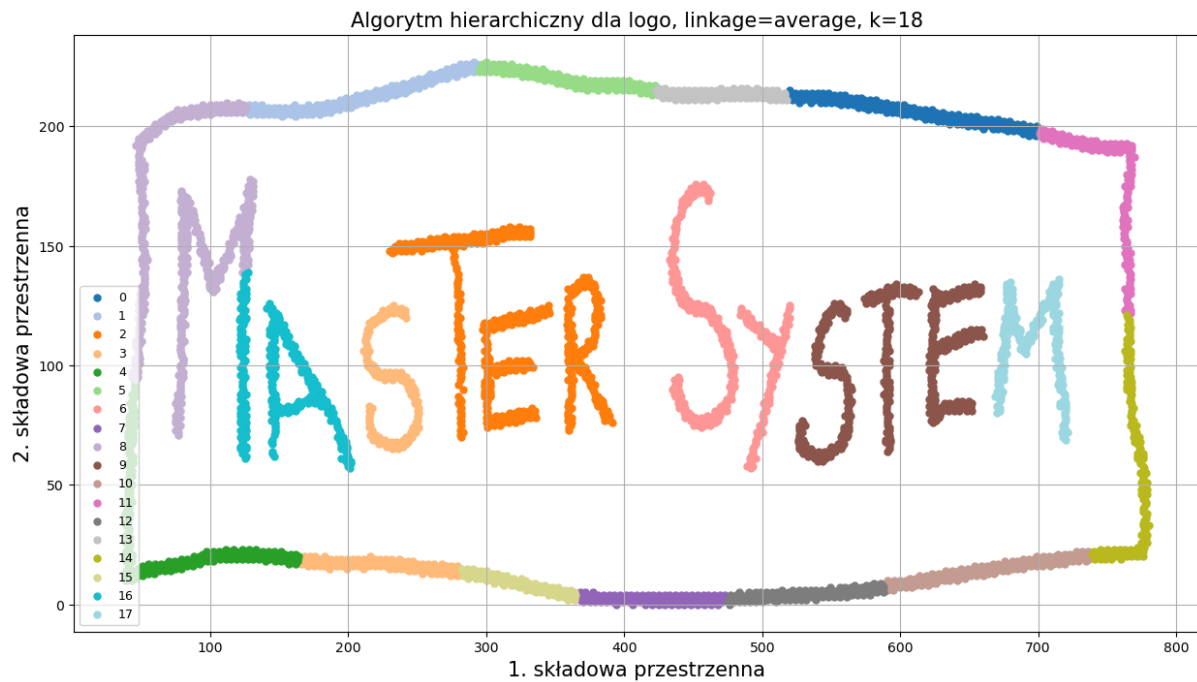
Zadanie 1

Obecne zbiory nastaw algorytmów klasyfikacji nienadzorowanej nie okazały się zbyt skuteczne w realizacji zadania segmentacji znaków w przykładowym, ręcznie narysowanym logu. Poszukaj lepszych nastaw algorytmów i postaraj się możliwie najdokładniej rozdzielić litery i ramkę loga poprzez zaklasyfikowanie ich do oddzielnych klas za pomocą badanych w przykładzie algorytmów.

1. Czy wszystkie algorytmy poradziły sobie równie dobrze w zadaniu segmentacji pisma? Jakie problemy z segmentacją pisma udało Ci się zaobserwować?
2. Jeśli jakiś algorytm nie poradził sobie w zadowalającym stopniu z tym zadaniem, dlaczego mogło się tak stać? Podobnie w przypadku algorytmów, którym udało się dokonać segmentacji - dlaczego akurat taki dobór nastaw okazał się najbardziej skuteczny?
3. Czy wygodniejsze do zadania segmentacji pisma są algorytmy wymagające znajomości liczby klastrów, czy te bazujące na gęstości punktów? Czy może istnieją Twoim zdaniem zastosowania, w których jedno z nich spisuje się lepiej i na odwrót?

Wykresy





Wnioski

- w takiego rodzaju danych, które nie są typowo skupione wokół centrum najlepiej poradził sobie DBSCAN. Dla $\text{eps}=8$ i $\text{min_samples}=3$ powstało 13 klastrów, czyli idealnie 1 ramka + 12 liter z napisu
- KMeans nie radzi sobie z tymi danymi, bo on się sprawdza lepiej przy danych bardziej sferycznych, a to są literki + ramka, a więc dane które są zupełnie innego układu; nawet przy większym $k=18$ KMeans nadal zaznacza punkty według położenia w przestrzeni. Ramka jest pokrojona na fragmenty, a część liter też dostaje sztuczne podziały. Nie działa to zbyt dobrze

- Algorytm hierarchiczny z `linkage='average'` i `k=18` wygląda lepiej, ale wciąż podział jest niepoprawny; algorytm hierarchiczny potrafi rozdzielać obszary, ale nie rozumie na przykład, że cała ramka jest jednym obiektem, albo zaznacza kilka literek itd.
- Cały czas chodzi o problem kształtu danych. KMeans i algorytm hierarchiczny muszą dostać liczbę klastrów, a do tego mocno patrzą na odległości globalne. DBSCAN patrzy lokalnie na gęstość i połączenia między punktami, więc dla oddzielonych kresek jest dużo wygodniejszy.
- Dla takich obrazków, gdzie znaki są osobnymi ciągłymi plamami, algorytm gęstościowy jest praktyczniejszy. Gdyby jednak znaki się stykały albo miały bardzo różną grubość kreski, DBSCAN też mógłby się pogubić i wtedy trzeba by dodać przetwarzanie obrazu przed klasteryzacją.

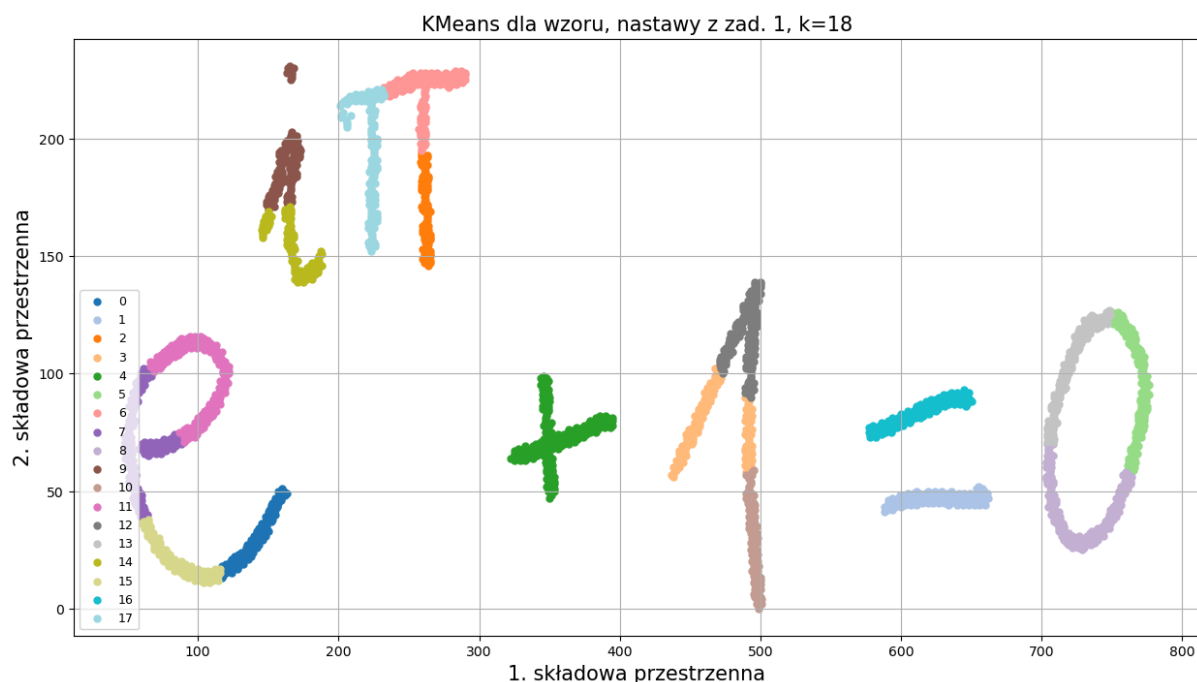
Zadanie 2

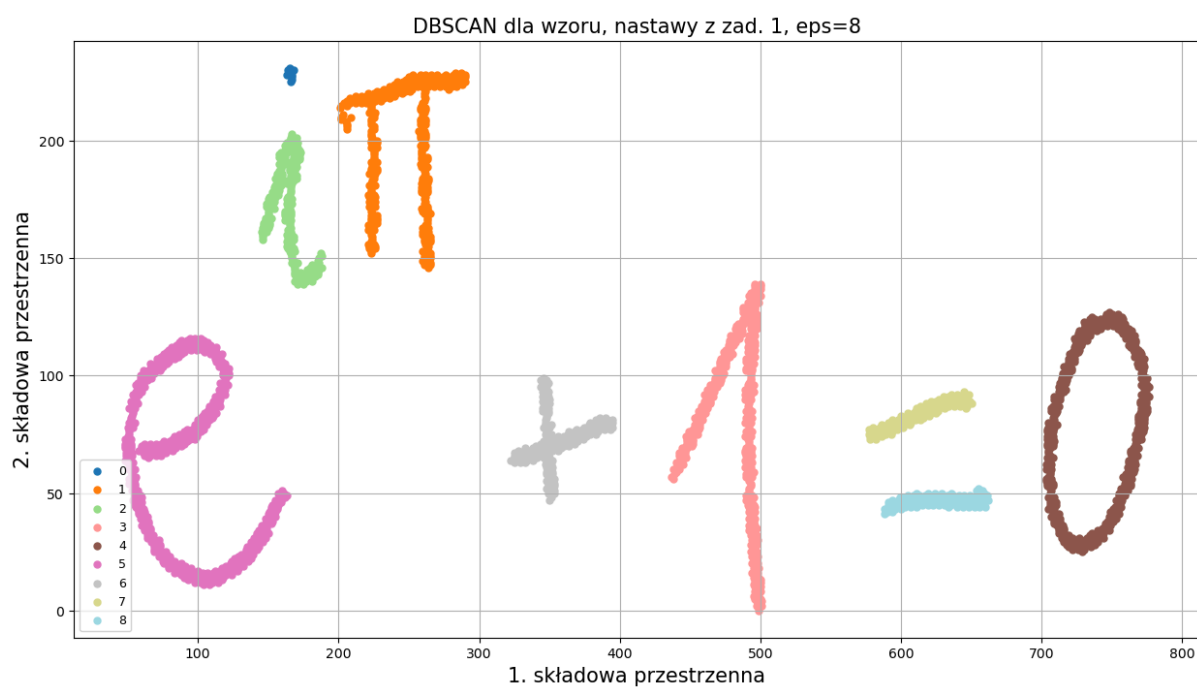
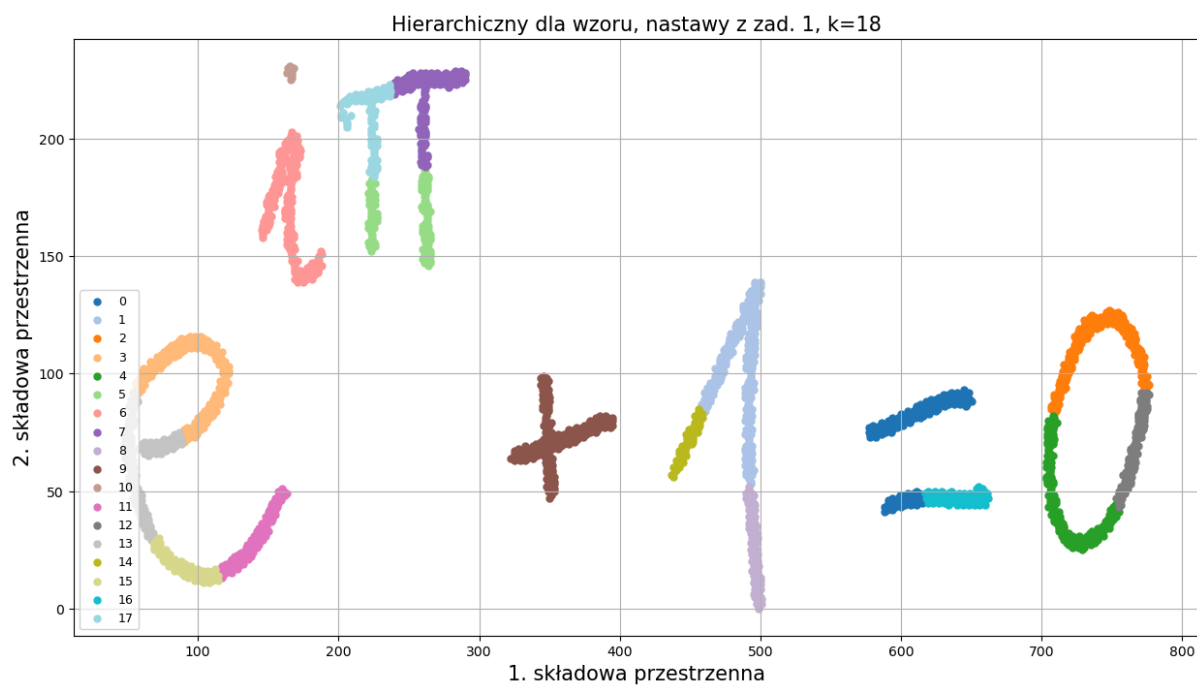
Zamień teraz plik wejściowy na plik "wzor_matematyczny.png". Uruchom ponownie (pozostawiając nastawy algorytmów, wprowadzone przez Ciebie w zadaniu 1.) wszystkie algorytmy klasyfikacji nienadzorowanej. Następnie, jeżeli któryś z algorytmów wymaga poprawienia nastaw, zmień je tak, aby znowu uzyskać rozdzielenie liter na osobne klasy.

- Czy był taki algorytm, który nie wymagał zmiany nastaw aby poprawnie dokonać segmentacji pisma?
- Jakie nowe problemy pojawiły się w przypadku segmentacji symboli matematycznych? Jakie poprawki lub jakie dodatkowe przetwarzanie należy jeszcze dodać, jeżeli efekt segmentacji z ćwiczenia miałby posłużyć np. do segmentacji znaków na potrzeby systemu rozpoznawania liter (OCR)?
- Jakie zalety i wady ma Twoim zdaniem algorytm DBSCAN, jeśli porównać go z algorytmem k-średnich i hierarchicznym?

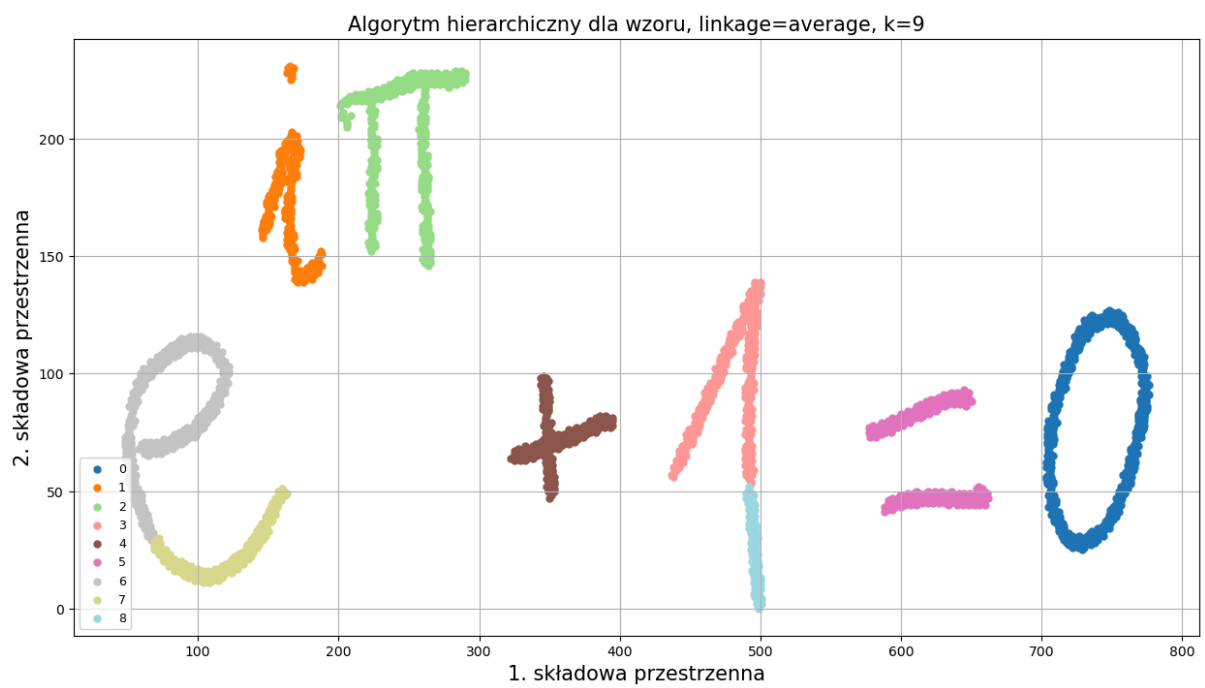
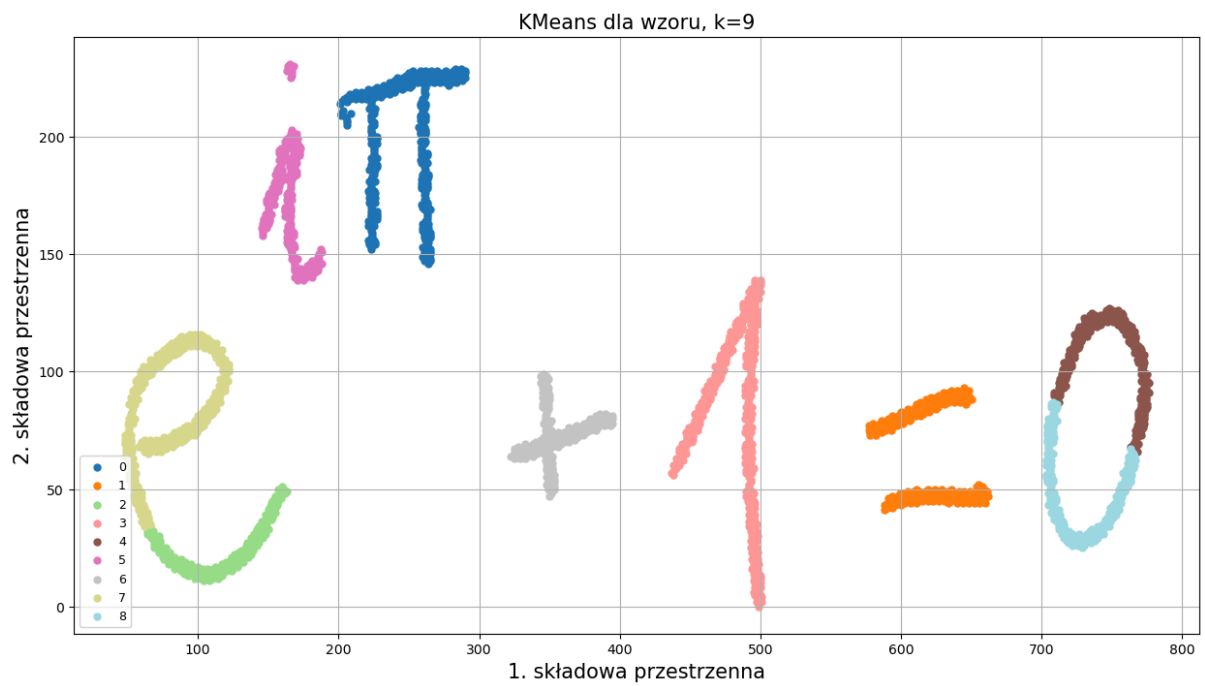
Wykresy

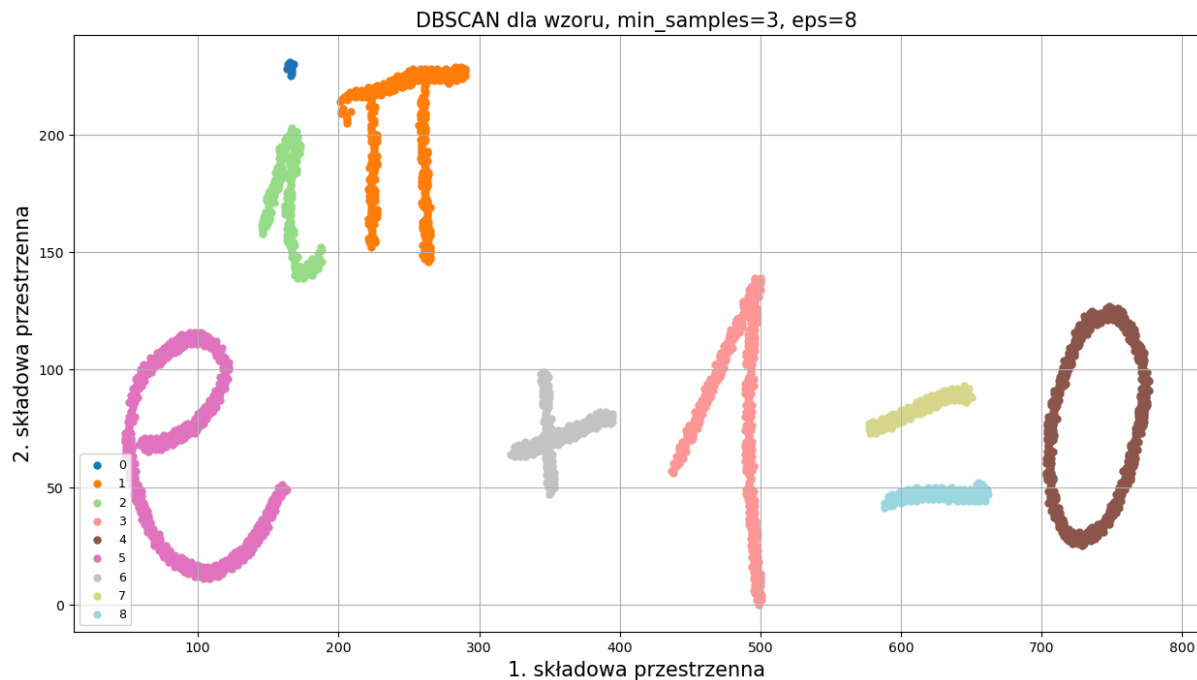
Przed zmianami:





Po zmianach:





Wnioski

- Dla wzoru matematycznego bez zmiany nastaw najlepiej poradził sobie **DBSCAN**. Parametry z loga nadal dały sensowny podział.
- KMeans i algorytm hierarchiczny trzeba było zmniejszyć do około $k=9$, bo przy $k=18$ dzieliły pojedyncze symbole na fragmenty. Po zmianie jest lepiej, ale dalej nie jest to idealne rozpoznawanie znaków, tylko segmentacja punktów.
- Nowy problem jest taki, że nie każdy znak jest jedną ciągłą kreską. Kropka nad i jest osobnym elementem, a znak $=$ składa się z dwóch oddzielnych kresek. DBSCAN naturalnie rozdziela takie części, więc do OCR trzeba by potem dodać etap łączenia bliskich komponentów w jeden symbol.
- Przydałoby się też wstępne czyszczenie obrazu, usuwanie szumu, normalizacja grubości kreski i analiza położenia elementów względem siebie, np. żeby wiedzieć, że kropka należy do i , a dwie równoległe kreski tworzą $=$.
- DBSCAN ma tę zaletę, że nie trzeba z góry znać liczby klastrów i bardzo dobrze łapie oddzielone komponenty. Wadą jest czułość na ϵ oraz to, że rozdziela znaki wieloczęściowe. KMeans i hierarchiczny są prostsze do kontrolowania przez k , ale przy piśmie ręcznym często tną obraz dość sztucznie.

Notatnik P4 (2 pkt) - wykorzystanie algorytmu k średnich do segmentacji obrazów

Zadanie 1

Kod

```
# Segmentacja/posteryzacja obrazu algorytmem k średnich
```

```

clusterer = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
clusters = clusterer.fit_predict(data)

# Nowy obrazek ma wysokość, szerokość i 3 kanały RGB.
segmented_image = np.zeros((image.size[1], image.size[0], 3),
dtype=np.uint8)

for cluster_id in np.unique(clusters):
    mask = clusters == cluster_id
    mean_color = image_df.loc[mask, ['vec_red', 'vec_green',
'vec_blue']].mean().to_numpy()
    rows = image_df.loc[mask, 'y_indices'].to_numpy()
    cols = image_df.loc[mask, 'x_indices'].to_numpy()
    segmented_image[rows, cols, :] = mean_color.astype(np.uint8)

print('Liczba utworzonych klastrów:', len(np.unique(clusters)))
print('Kolory średnie klastrów RGB:')
for cluster_id in np.unique(clusters):
    mask = clusters == cluster_id
    mean_color = image_df.loc[mask, ['vec_red', 'vec_green',
'vec_blue']].mean().round(1).to_numpy()
    print(f'klaster {cluster_id}: {mean_color}')

plt.figure(figsize=(14, 7))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title('Obraz oryginalny', fontsize=15)
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(segmented_image)
plt.title(f'Obraz po posteryzacji, k={n_clusters}', fontsize=15)
plt.axis('off')

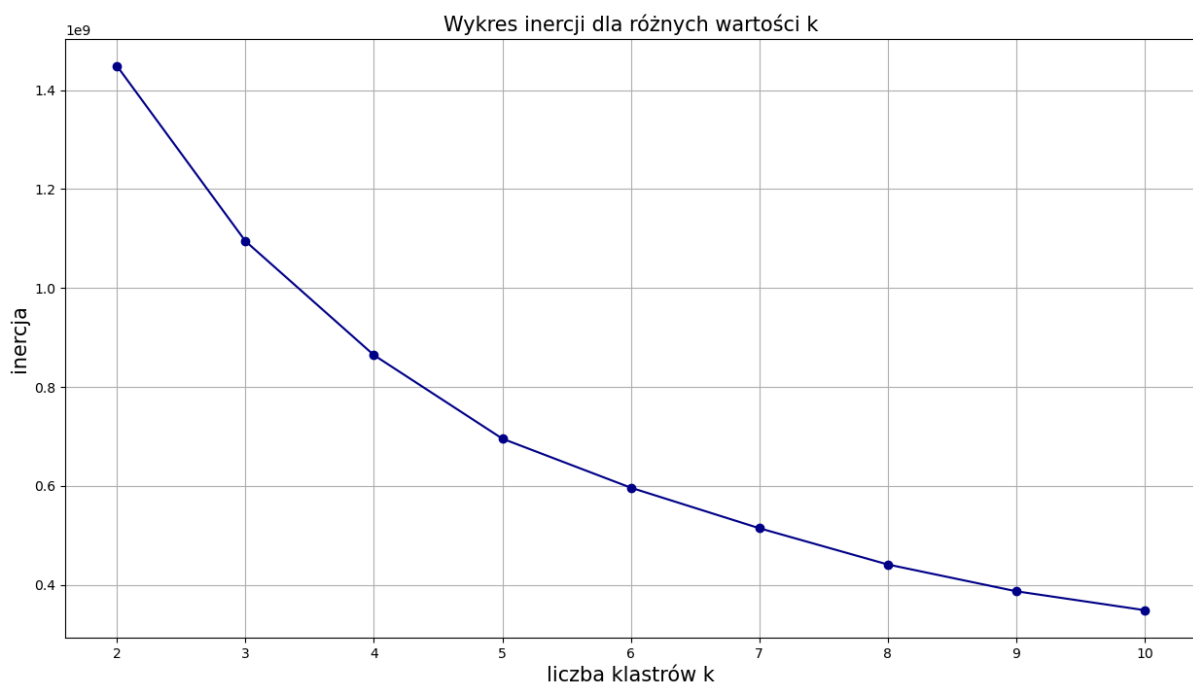
```

Wykres

Obraz oryginalny



Obraz po posteryzacji, k=6



Wnioski

- Po zastosowaniu KMeans obraz został uproszczony do kilku dużych grup kolorów. Przy $k=6$ kwiat, niebo, ciemne tło i liście są nadal rozpoznawalne, ale przejścia już nie ma. Nawet ładny efekt / ciekawy.
- Wykres inercji nie daje idealnie ostrego łokcia, ale po $k=5$ / 6 / 7 spadek robi się już zauważalnie wolniejszy. Dalsze zwiększanie k wciąż poprawiałoby szczegóły, ale efekt posteryzacji byłby mniej wyraźny.
- Takie przetwarzanie może się przydać do redukcji liczby kolorów, kompresji, szybkiego wydzielania większych obszarów obrazu. W praktyce w grafice

komputerowej można by czegoś w tym stylu użyć do oddzielenia nieba od obiektu, uproszczenie tła, przygotowanie grafiki w stylu plakatu.

- Minusem jest to, że KMeans realnie nie rozumie co jest na obrazie. Klaster może oznaczać podobny kolor w różnych miejscach, a niekoniecznie jeden konkretny obiekt. Czyli do poważnej segmentacji trzeba by dołożyć jeszcze inne cechy albo inny algorytm / podejście